

Reviewer Pack — Minimal Rank of Noncrossing Partitions over Matroids vs Comm...

Entry a9a1b2211d7b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 11:40:08 UTC

Minimal Rank of Noncrossing Partitions over Matroids vs Communication Complexity for Disjointness

Entry ID: a9a1b2211d7b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 11:40:08 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics **Field B** (complexity object): Communication Complexity: Disjointness

Statement:

[‘For every instance of the disjointness problem with n variables, there exists a noncrossing partition N on the matroid associated with the instance such that the rank of N is lower bounded by $\Omega(n)$.’, ‘Specifically, $\text{rank}(N) = \Omega(n)$ for all instances with communication complexity at least $\Omega(n)$.’, ‘Furthermore, if the disjointness problem can be solved with communication complexity $C(n)$, then there exists a noncrossing partition N on its associated matroid such that $\text{rank}(N) \leq C(n)$.’]

Rationale (proposer’s reasoning):

[‘Noncrossing partitions provide a rich combinatorial structure that may capture essential properties of the underlying matroids, which in turn might relate to communication complexity.’, ‘The algebraic nature of noncrossing partitions allows for potential applications of algebraic techniques to analyze communication complexity problems.’, ‘This conjecture would bridge algebraic combinatorics with communication complexity, potentially leading to new insights and algorithms.’]

Taxonomy category: COMM_DISJ (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 03798296ae57304a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If for all instances, the rank of the noncrossing partition is at least $\Omega(n)$ and no instance with rank less than $\Omega(n)$ has a communication complexity below $\Omega(n)$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "noncrossing partition" AND "matroid" AND "communication complexity" AND "disjointness" - "rank of partition" IN matroids AND communication complexity AND disjointness - "algebraic combinatorics" AND "communication complexity" AND minimal rank noncrossing partitions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_disjointness_instance(n):
        return [random.randint(0, 1) for _ in range(n)]

    def compute_matroid(instance):
        matroid = {}
        for i in range(len(instance)):
            if instance[i] == 1:
                matroid[i] = set(range(i + 1))
        return matroid

    def noncrossing_partitions(matroid, n):
        partitions = []
        for i in range(1 << n):
            partition = [[]]
            for j in range(n):
                if i & (1 << j):
                    partition[-1].append(j)
                else:
                    partition.append([j])
            partitions.append(partition)
        return partitions

    def rank_of_partition(partition, matroid):
```

```

max_rank = 0
for block in partition:
    block_set = set(block)
    if all(block_set.issubset(matroid[i]) for i in block):
        max_rank = max(max_rank, len(block))
return max_rank

def communication_complexity(instance):
    n = len(instance)
    count_ones = instance.count(1)
    return 2 * count_ones - 1 if count_ones > 0 else 0

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    instance = generate_disjointness_instance(n)
    matroid = compute_matroid(instance)
    partitions = noncrossing_partitions(matroid, n)

    for partition in partitions:
        rank = rank_of_partition(partition, matroid)
        comm_complexity = communication_complexity(instance)
        results.append({
            "n": n,
            "instance": instance,
            "matroid": matroid,
            "partition": partition,
            "rank": rank,
            "comm_complexity": comm_complexity
        })

if not results:
    return {
        "metric_name": "rank",
        "metric_value": 0,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

rank_values = [result["rank"] for result in results]
comm_complexity_values = [result["comm_complexity"] for result in results]

mean_rank = sum(rank_values) / len(rank_values)
std_rank = math.sqrt(sum((x - mean_rank) ** 2 for x in rank_values) / len(rank_values))
support_fraction = sum(1 for r, cc in zip(rank_values, comm_complexity_values) if r >= n and cc <= r)

return {
    "metric_name": "rank",
    "metric_value": mean_rank,
    "instances_tested": len(results),
    "conjecture_holds": support_fraction == 1.0,
    "counterexample": "" if support_fraction == 1.0 else f"support_fraction={support_fraction}"
}

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction == 1.0:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"support_fraction={support_fraction}\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5f536569.py", line 107, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5f536569.py", line 65, in run_trial
    rank = rank_of_partition(partition, matroid)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5f536569.py", line 47, in rank_of_partition
    if all(block_set.issubset(matroid[i]) for i in block):
       ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5f536569.py", line 47, in <genexpr>
    if all(block_set.issubset(matroid[i]) for i in block):
       ~~~~~^
KeyError: 3

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means we cannot verify the conjecture's conditions. | next: Re-run the test to ensure it completes successfully and provides results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14569
2	preregistration	ollama_remote	glm4:latest	0	0	9028
3	novelty	ollama_remote	glm4:latest	0	0	8880
4	novelty	ollama_remote	glm4:latest	0	0	9061
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12350
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9734
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7614
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12169
9	judge	ollama_remote	glm4:latest	0	0	11286

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 94692 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/a9a1b2211d7b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a9a1b2211d7b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a9a1b2211d7b.tar.gz` (if generated)